Анимации

Анимации

Проблема

Допустим, у нас есть персонаж. Пока мы разбирались с архитектурой, автоматами и камерой, он был просто цветным прямоугольником. Ну, может, статичной картинкой. Стоит на месте - одна картинка. Бежит - та же картинка. Атакует - всё та же картинка. Выглядит как искусство.

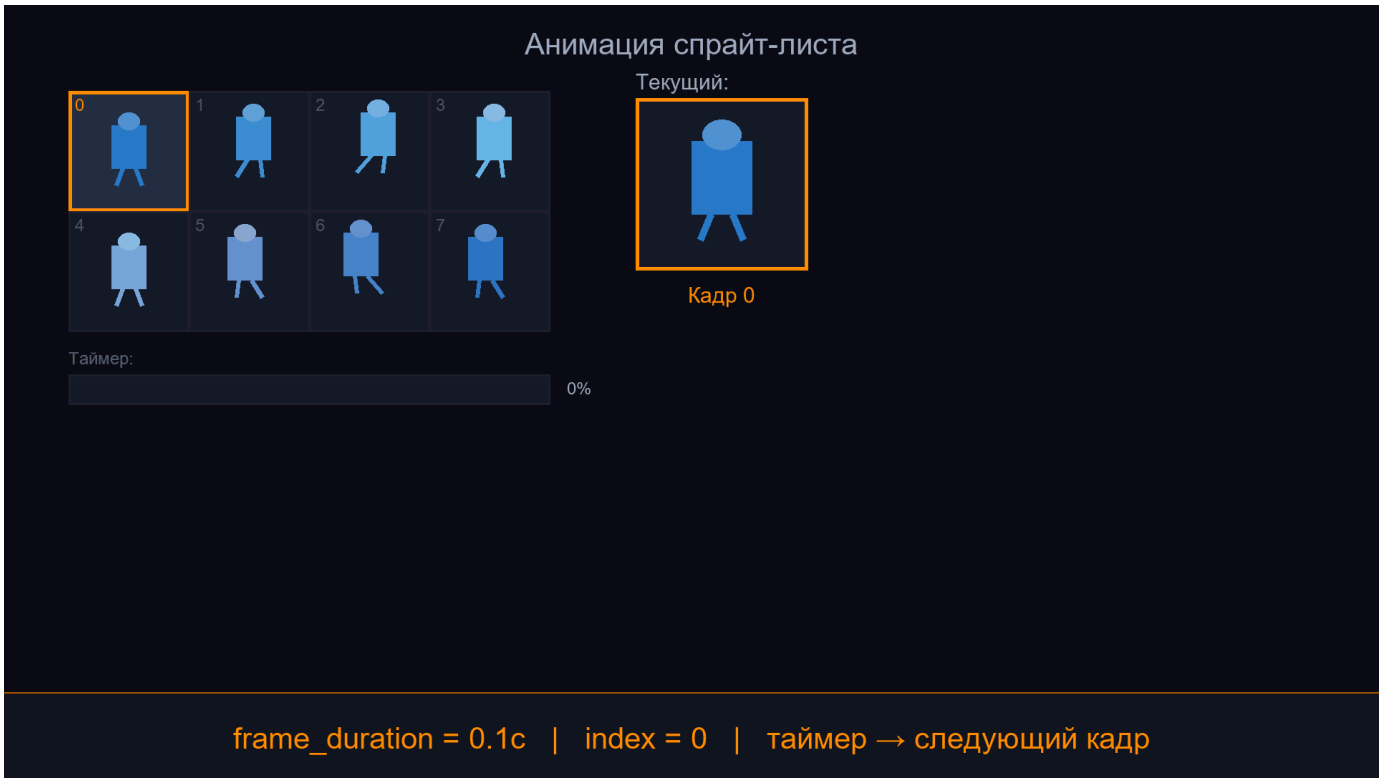
Нам нужно, чтобы персонаж выглядел живым: ноги двигаются при ходьбе, руки машут при атаке, тело покачивается в покое. Как этого добиться?

Принцип тот же, что и в мультфильмах: быстро показываем последовательность картинок, и глаз воспринимает это как движение. Каждая картинка - это **кадр** (frame) анимации. Показываем 8-12 кадров в секунду - и прямоугольник превращается в живого персонажа.

Спрайт-лист

Первая мысль: хранить каждый кадр в отдельном файле. `walk_01.png`, `walk_02.png`, ... `walk_08.png`. Это работает, но неудобно - 10 анимаций по 8 кадров = 80 файлов. Загрузка каждого файла - это обращение к диску, и на слабых машинах это может заметно тормозить.

Поэтому используют **спрайт-лист** (sprite sheet) - одно большое изображение, в котором все кадры расположены сеткой:



Каждый кадр имеет одинаковый размер (например, 64x64 пикселей). Чтобы получить нужный кадр, мы вычисляем его положение на листе по индексу. Т.е. зная номер кадра, размер кадра и количество столбцов, мы можем вырезать прямоугольную область из большого изображения.

Python | C#

```python
class SpriteSheet:
    def __init__(self, image, frame_width, frame_height):
        self.image = image
        self.frame_width = frame_width
        self.frame_height = frame_height
        self.columns = image.get_width() // frame_width

    def get_frame(self, index):
        col = index % self.columns
        row = index // self.columns
        x = col * self.frame_width
        y = row * self.frame_height
        frame = self.image.subsurface((x, y, self.frame_width, self.frame_h
        return frame


# usage
sheet_image = pygame.image.load("hero_walk.png").convert_alpha()
sheet = SpriteSheet(sheet_image, frame_width=64, frame_height=64)
```

```
frame_0 = sheet.get_frame(0)  # first frame
frame_5 = sheet.get_frame(5)  # sixth frame
```

Спрайт-листы можно собирать вручную в графическом редакторе или использовать инструменты вроде TexturePacker, Aseprite (экспорт), или LibreSprite (бесплатный). Многие бесплатные ассеты на itch.io уже идут в виде готовых спрайт-листов.

Переключение кадров

Ок. Мы умеем доставать отдельные кадры из листа. Теперь нужно их переключать - показывать один за другим, чтобы получилась анимация.

Наивный подход: менять кадр на каждой итерации игрового цикла. Каждый `Update` - следующий кадр.

Проблема? Скорость анимации привязана к FPS. При 60 FPS анимация из 8 кадров проиграется за 0.13 секунды - слишком быстро, персонаж мельтешит. При 30 FPS - за 0.27 секунды. На разных компьютерах анимация будет выглядеть по-разному. Знакомая ситуация - мы уже сталкивались с этим в игровом цикле, когда обсуждали deltaTime.

Решение то же: считаем время. Заводим таймер, и переключаем кадр только когда прошло до

Python C#

```
class Animation:
    def __init__(self, sprite_sheet, frames, frame_duration):
        self.sprite_sheet = sprite_sheet
        self.frames = frames            # list of frame indices, e.g. [0, 
        self.frame_duration = frame_duration  # seconds per frame
        self.current_index = 0
        self.timer = 0.0

    def update(self, delta_time):
        self.timer += delta_time
        if self.timer >= self.frame_duration:
            self.timer -= self.frame_duration
            self.current_index = (self.current_index + 1) % len(self.frame

    def get_current_frame(self):
        frame_id = self.frames[self.current_index]
        return self.sprite_sheet.get_frame(frame_id)
```

```
# usage
walk_anim = Animation(sheet, frames=[0, 1, 2, 3, 4, 5], frame_duration=0.1)

# in game loop
walk_anim.update(delta_time)
current_frame = walk_anim.get_current_frame()
screen.blit(current_frame, (player.x, player.y))
```

`frame_duration` задаётся в **секундах**. При `frame_duration = 0.1` кадр меняется каждые 100 мс, т.е. 10 кадров анимации в секунду. Это не зависит от FPS игры - на 30 FPS и на 120 FPS анимация будет выглядеть одинаково.

Обратите внимание: `timer -= frame_duration`, а не `timer = 0` - так мы не теряем остаток времени при лагах.

Несколько анимаций: автомат анимаций

У персонажа обычно не одна анимация. Как минимум: покой (idle), ходьба (walk), прыжок (jump), атака (attack). Как переключаться между ними?

Если вы читали раздел про конечные автоматы, то ответ уже знаете. Анимации - это состояния, а переходы между ними зависят от того, что делает персонаж. Стоит - idle. Пошёл - walk. Нажал атаку - attack. Анимация атаки закончилась - обратно в idle (или walk, если всё ещё движется).

Не обязательно строить полноценный автомат с классами состояний. Для анимаций часто хватает простого подхода: храним словарь анимации и текущее состояние. Ключи - **enum**, не строки. Опечатка в строке `"atack"` - тихий баг, а `AnimState.ATACK` не скомпилируется.

Python | C#

```python
from enum import Enum

class AnimState(Enum):
    IDLE = 0
    WALK = 1
    ATTACK = 2


class AnimationController:
    def __init__(self):
        self.animations = {}
        self.current_state = None
        self.current_anim = None

    def add(self, state, animation):
```

```
        self.animations[state] = animation

    def play(self, state):
        if self.current_state == state:
            return  # already playing
        self.current_state = state
        self.current_anim = self.animations[state]
        self.current_anim.reset()

    def update(self, delta_time):
        if self.current_anim:
            self.current_anim.update(delta_time)

    def get_current_frame(self):
        if self.current_anim:
            return self.current_anim.get_current_frame()
        return None


# setup
controller = AnimationController()
controller.add(AnimState.IDLE, Animation(sheet, frames=[0, 1], frame_durat:
controller.add(AnimState.WALK, Animation(sheet, frames=[2, 3, 4, 5, 6, 7],
controller.add(AnimState.ATTACK, Animation(sheet, frames=[8, 9, 10, 11], fı
controller.play(AnimState.IDLE)

# in game loop
if is_attacking:
    controller.play(AnimState.ATTACK)
elif is_moving:
    controller.play(AnimState.WALK)
else:
    controller.play(AnimState.IDLE)

controller.update(delta_time)
frame = controller.get_current_frame()
screen.blit(frame, (player.x, player.y))
```

Метод `play` проверяет, не играет ли уже эта анимация. Если да - ничего не делает, если нет - сбрасывает в начало. Без этой проверки каждый кадр перезапускает анимацию - будет дёрганье.

Для этого добавим метод `reset` в наш класс `Animation` :

Python C#

```python
def reset(self):
    self.current_index = 0
    self.timer = 0.0
```

Одноразовые анимации

Ходьба и покой - циклические анимации. Они крутятся по кругу, пока персонаж делает соответствующее действие. Но атака - это разовое действие. Анимация атаки должна проиграться один раз и остановиться (или переключиться на другую).

Для этого добавим в `Animation` флаг **looping** и способ узнать, закончилась ли анимация.

Теперь в игровом цикле проверяем `finished` - если анимация закончилась, переключаемся на `idle`. Грубо говоря: "Играй атаку. Когда закончится - вернись в покой." Если вы строили автомат для персонажа, то анимация атаки - это состояние с автоматическим переходом по завершении.

Отражение спрайтов

Персонаж идёт вправо - показываем анимацию ходьбы. Персонаж идёт влево - показываем... ту же анимацию, но отражённую по горизонтали. Зачем рисовать отдельные спрайты для каждого направления, если можно просто перевернуть?

Python C#

```python
# flip horizontally
flipped_frame = pygame.transform.flip(frame, True, False)

# in practice:
frame = controller.get_current_frame()
if facing_left:
    frame = pygame.transform.flip(frame, True, False)
screen.blit(frame, (player.x, player.y))
```

В Python мы создаём отражённую копию через `pygame.transform.flip`. В MonoGame используем `SpriteEffects.FlipHorizontally` - это флаг для `Draw`, текстура при этом не копируется.

Для четырёх направлений (top-down игры) отражение не поможет - спрайт "вверх" не получить из спрайта "вниз". В таких играх в спрайт-листе хранят отдельные ряды для каждого направления: ряд 0 - вниз, ряд 1 - влево, ряд 2 - вправо, ряд 3 - вверх.

Привязка событий к кадрам

Иногда нужно, чтобы что-то произошло на конкретном кадре анимации. Например, звук удара должен раздаться не в начале анимации атаки, а в момент, когда меч достигает цели - допустим, на 3-м кадре. Или проверка коллизии атаки должна случиться на определённом кадре, а не на протяжении всей анимации.

Простой способ: после каждого переключения кадра проверяем, не является ли новый кадр "событийным":

Это пересекается с событийной моделью: анимация - издатель, а звуковая система и система коллизий - подписчики. Можно вместо колбэков использовать полноценные события, если у вас уже есть такая система.

Итого

Анимация в 2D - это:

1. **Спрайт-лист** - все кадры в одном изображении.
2. **Таймер на deltaTime** - переключение кадров не зависит от FPS.
3. **Контроллер анимаций** - словарь анимаций + переключение между ними.
4. **Looping** - циклические (idle, walk) и одноразовые (attack, death) анимации.
5. **Отражение** - один набор спрайтов для двух направлений.

Что	Где почитать подробнее
deltaTime и игровой цикл	Основной цикл
Конечные автоматы для состояний персонажа	Конечные автоматы
Коллизии и хитбоксы для атак	Коллизии
Событийная модель для привязки звуков/эффектов	Событийная модель
Типичные ошибки (логика в Draw, нет deltaTime)	Типичные ошибки

Powered by Wiki.js